

AI Brand Voice Generator

Creating Consistent Marketing Content with Gemini

The AI Brand Voice Generator is an intelligent web application powered by Google's Gemini generative AI models that creates brand-consistent marketing content instantly. It allows marketers, founders, and creators to generate social media posts, emails, taglines, ad headlines, and blog intros all in their unique brand tone and style.

This system bridges the gap between creativity and automation by understanding a brand's tone, vocabulary, rhythm, and emotional attitude from sample texts provided by the user, and then reproducing that same brand voice in fresh, original marketing content.

Scenario 1: Marketing and Content Creation

Marketers and business owners can use the AI Brand Voice Generator to produce tailored marketing content quickly without compromising on brand tone or quality.

By providing a few examples of their existing marketing materials such as taglines, emails, or posts users enable the model to **learn their unique communication style**. Once trained on this tone, the system can generate complete marketing copy for different platforms.

For instance, a startup launching a new product can upload a few sample taglines from past campaigns. The AI will analyse those texts, understand the emotional and linguistic patterns, and generate brand-consistent content such as a catchy Instagram post or an engaging product announcement email all within seconds.

This helps teams **save time**, maintain **consistent brand identity**, and **scale marketing efforts** effortlessly.

Scenario 2: Personalized Branding and Creative Assistance

Freelancers, content strategists, and agencies can use this system as a creative assistant for generating ideas or testing tones across multiple brands.

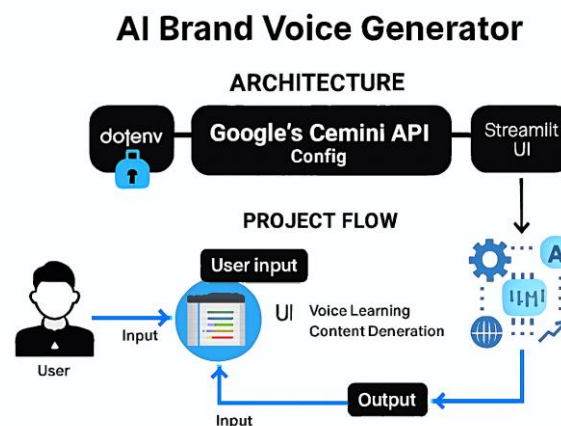
By analysing sample content from different clients, the AI Brand Voice Generator can automatically switch between voices from playful to professional, emotional to technical allowing professionals to manage multiple brand personalities effectively.

For instance, a marketing agency managing both a lifestyle brand and a tech company can input each brand's sample text separately, and the AI will generate customized materials that reflect each brand's tone perfectly.

This use case demonstrates how the system can help creative professionals maintain authenticity, reduce manual writing time, and enhance creative output.

Architecture

The AI Brand Voice Generator uses a modular architecture that integrates Google's Gemini AI, Streamlit UI, and environment management via dotenv for secure configuration.



Project Flow:

1. **User Input:** Users enter brand name, campaign purpose, and provide sample brand texts in the Streamlit interface.
2. **Voice Learning:** The application calls the `learn_brand_voice()` function, which constructs a structured prompt that teaches the Gemini model the brand's tone, emotion, and vocabulary.
3. **Content Generation:** Once the brand voice is learned, the user selects the desired content type (e.g., social post, email, tagline). The system then passes this prompt to `generate_brand_content()`, which uses the Gemini API to generate relevant, brand-consistent content.

4. **Output Display:** The generated text is displayed in the Streamlit app, where users can view, copy, and download the results.
5. **Deployment:** The app is deployed via Streamlit, allowing real-time content generation in a web environment.

To Accomplish This Project, You Must Complete the Following Activities:

- Install and configure all required libraries and APIs.
- Initialize the Gemini model.
- Develop the brand learning and content generation functions.
- Build the Streamlit user interface.
- Deploy the complete application using Streamlit.

Prior Knowledge

You should have an understanding of:

1. Google Generative AI (Gemini)

Google Gemini provides advanced generative AI models for natural language understanding and content generation. Knowledge of interacting with the Gemini API, configuring models, and sending structured prompts is essential.

Official Documentation:

https://cloud.google.com/gemini/docs?utm_source=chatgpt.com

2. Streamlit

Streamlit is a Python-based framework for building interactive web applications. Understanding widgets, forms, layout management, and state handling is crucial for building the frontend.

Official Tutorials:

https://docs.streamlit.io/develop/tutorials?utm_source=chatgpt.com

3. Environment Management

The python-dotenv library allows secure management of environment variables such as API keys. Ensures sensitive credentials are not exposed in your code.

Official Documentation:

https://pypi.org/project/python-dotenv/?utm_source=chatgpt.com

4. Prompt Engineering

Prompt engineering is the process of designing and structuring input prompts for AI models to produce accurate, high-quality output. Knowledge of crafting clear instructions, templates, and examples helps the AI understand context, tone, and desired output format.

Learning Resources:

<https://platform.openai.com/docs/guides/completion/prompt-design>

<https://www.datacamp.com/blog/prompt-engineering>

5. Python Programming

Core Python skills are essential for integrating the AI model, handling data, and building backend logic. Knowledge of libraries like os, pyperclip, typing, and API interaction using Python is required.

Official Documentation:

<https://docs.python.org/3/>

Project Structure

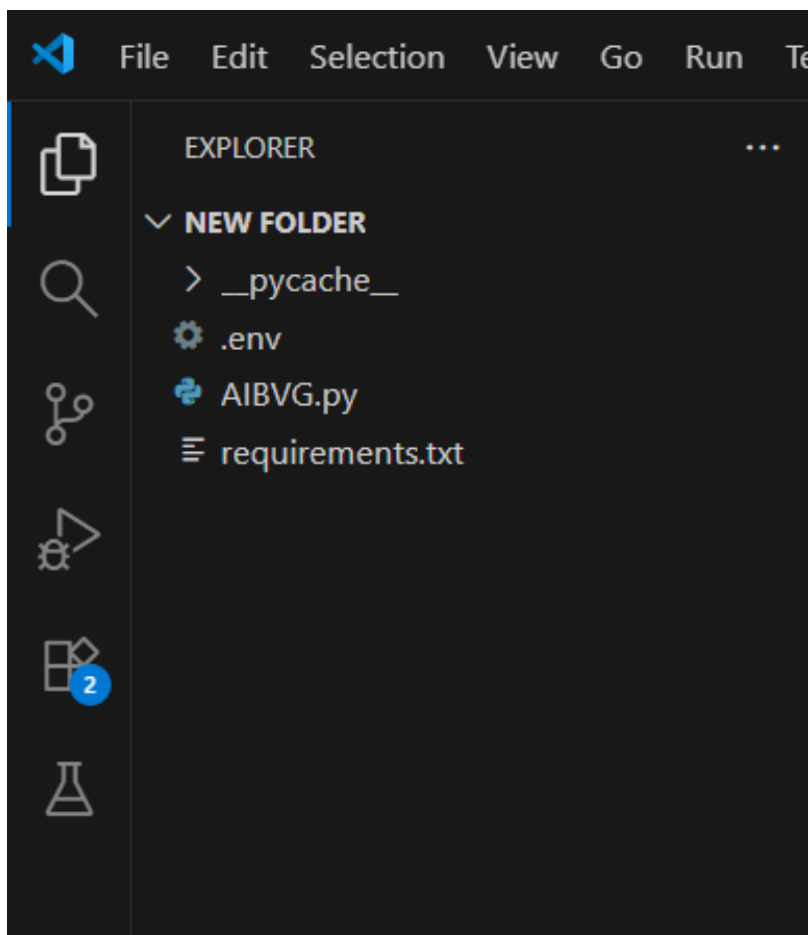
This project can be developed on VS code editor with Python 3.11.3.

The final structure looks like this:

📁 AI_Brand_Voice_Generator

- ├ 📄 app.py → Main application and Streamlit UI
- ├ 📄 .env → Stores the GEMINI_API_KEY securely
- └ 📄 requirements.txt → Lists dependencies

Picture of File structure from VS code editor:



Milestone 1: Requirements Specification

Milestone: Library Installation and Setup

This milestone ensures all **necessary libraries** are installed and configured to run the AI Brand Voice Generator app. It prepares the environment for secure API access, interactive web interfaces, and efficient content handling.

Option 1: Install Required Libraries

The following libraries are needed:

1. **Streamlit** – For creating interactive AI and data science web applications.
2. **google-generativeai** – Official library to interact with **Google's Gemini API** for content generation.
3. **python-dotenv** – To securely manage and load API keys from .env files.
4. **pyperclip** – Enables copying generated content directly to the clipboard.

Installation Command:

```
pip install streamlit google-generativeai python-dotenv pyperclip
```

Option 2: Using requirements.txt for Library Installation

This approach ensures **reproducibility and easy setup** for all users by installing all required libraries at once from a requirements.txt file.

Steps

1. Ensure requirements.txt Exists

The project folder should contain a requirements.txt file with the following content:

- streamlit
- google-generativeai
- python-dotenv
- pyperclip

2. Install All Libraries

Run the following command in your project directory to install all dependencies at once:

```
pip install -r requirements.txt
```

Explanation of Each Library

- **Streamlit:** Provides a simple, fast way to build interactive web apps for AI and data science projects without front-end coding.
- **google-generativeai:** Connects to Google Gemini to generate high-quality, brand-aligned content.
- **python-dotenv:** Loads sensitive environment variables like API keys securely from .env files.
- **pyperclip:** Enables copying AI-generated content directly to the clipboard, improving user experience.

Milestone 2: Initializing the Models

The Gemini model is configured and initialized inside the main application file app.py.

Activity 1: Loading Environment Variables

This activity ensures secure access to the Gemini API by loading sensitive credentials from a hidden **.env** file instead of hardcoding them in the script.

Explanation

- **Loads environment variables:** Uses `load_dotenv()` to read values from the `.env` file.
- **Retrieves API key securely:** Fetches the `GEMINI_API_KEY` without exposing it in the code.
- **Validates configuration:** Alerts the user if the key is missing, preventing runtime errors and maintaining security.

Code snippet:

```
AIBVG.py > generate_brand_content
1 import streamlit as st
2 import google.generativeai as genai
3 from dotenv import load_dotenv
4 import pyperclip
5 import os
6
7 load_dotenv()
8 API_KEY = os.getenv("GEMINI_API_KEY")
9
```

Activity 2: Configuring and Initializing the Model

This activity initializes the Gemini 2.0 Flash model, enabling the app to generate fast, high-quality, and contextually accurate marketing content.

Explanation

- **Imports the Gemini library:** Provides access to Google’s generative AI features.
- **Configures API access:** Connects the app to Gemini using the secure API key.
- **Loads the model:** Initializes “gemini-2.0-flash”, optimized for rapid text generation and strong contextual understanding.

Code snippet:

```
AIBVG.py > generate_brand_content
1 import streamlit as st
2 import google.generativeai as genai
3 from dotenv import load_dotenv
4 import pyperclip
5 import os
6
7 load_dotenv()
8 API_KEY = os.getenv("GEMINI_API_KEY")
9
10 if API_KEY:
11     genai.configure(api_key=API_KEY)
12 else:
13     st.error("❌ No API key found. Please add it to your .env file.")
14
15 model = genai.GenerativeModel("gemini-2.0-flash")
16
```


Milestone 3: Building the Core Functions

This milestone covers the two key functional components brand voice learning and content generation.

Activity 1: Creating learn_brand_voice()

This activity builds a system prompt that helps Gemini learn and replicate a brand's unique communication style. By analysing user-provided samples, it captures the brand's tone, emotion, vocabulary, and rhythm to ensure future AI-generated content stays consistent and authentic.

Explanation

- **Collects sample texts:** Gathers user-provided examples that best reflect the brand's natural voice.
- **Analyses tone and emotion:** Instructs Gemini to identify the overall mood, attitude, and emotional resonance.
- **Examines vocabulary and rhythm:** Helps the model understand word choice, pacing, and sentence flow.
- **Extracts key features:** Captures recurring linguistic and stylistic traits that define the brand's identity.
- **Prepares for future use:** Enables Gemini to generate new content in the same consistent brand voice.

Code snippet:

```
16
17 # FUNCTIONS
18 def learn_brand_voice(sample_texts):
19     prompt = (
20         "You are an AI marketing writer. Analyze these samples to deeply learn their tone, "
21         "vocabulary, emotional style, and sentence rhythm.\n\n"
22     )
23     for i, text in enumerate(sample_texts, start=1):
24         prompt += f"Example {i}: {text.strip()}\n"
25     prompt += (
26         "\nFrom these examples, infer the following:\n"
27         "- Writing tone (e.g., formal, playful, confident)\n"
28         "- Emotional attitude (e.g., inspiring, friendly, authoritative)\n"
29         "- Vocabulary patterns (e.g., simple, descriptive)\n"
30         "- Sentence rhythm and perspective\n"
31         "\nUse this style in all future content generations. Do not output analysis unless asked."
32     )
33     return prompt
34
```

Activity 2: Creating generate_brand_content()

This activity focuses on generating customized marketing content using the brand voice learned earlier. It combines the brand's tone and style with user inputs like brand name, purpose, and content type to create consistent, high-quality outputs.

Explanation

- **Uses pre-defined templates:** Applies ready-made structures for different content types such as social posts, emails, or taglines.
- **Combines brand data with templates:** Integrates the learned brand voice into the selected template to maintain stylistic consistency.
- **Generates content with Gemini:** Sends the final combined prompt to the model for AI-powered generation.
- **Delivers brand-aligned output:** Returns polished, on-brand text ready for marketing use.

Code snippet:

```
35
36 def generate_brand_content(brand_prompt, brand_name, purpose, content_type):
37     topic = f"{brand_name} - {purpose}"
38
39 >     templates = { ...
156     content_template = templates.get(content_type, templates["Social Media Post"])
157     final_prompt = f"{brand_prompt}\n\n{content_template}"
158
159     try:
160         response = model.generate_content(final_prompt)
161         return response.text.strip()
162     except Exception as e:
163         return f"[Error] Failed to generate content: {str(e)}"
164
```

Milestone 4: Creating Streamlit Frontend

The app.py file also contains the Streamlit-based user interface.

Activity 1: UI Initialization

This activity sets up the Streamlit interface for the AI Brand Voice Generator app, defining its layout, title, and appearance.

Explanation

- **Configures page settings:** Sets the app's title, icon, and layout for a clean, wide-screen view.
- **Displays main heading:** Shows the project title *"AI Brand Voice Generator"* on the interface.
- **Introduces the app:** Provides users with a clear entry point and context before interacting with features.

Code snippet:

```
165 # STREAMLIT UI
166 st.set_page_config(page_title="AI Brand Voice Generator", layout="wide", page_icon="🗣️")
167
168 # MAIN PAGE
169 st.title("🗣️ AI Brand Voice Generator")
170 st.markdown("""
171 Welcome to the AI Brand Voice Generator – a powerful tool that helps you instantly create high-quality, brand-consistent marke
172
173 Provide a few examples of your brand's voice, and the AI will generate:
174 - 📱 Social media posts
175 - ✉️ Professional emails
176 - 📝 Blog intros
177 - 🏷️ Taglines & Ad Headlines
178
179 Perfect for marketers, founders, and content creators who want speed without losing the brand feel.
180 """)
```

Activity 2: Sidebar Inputs

The sidebar captures user information:

- Brand name
- Campaign purpose
- Sample brand texts
- Content type (social media, email, etc.)

These inputs are passed to the functions for processing and generation.

Code snippet:

```
182 # SIDEBAR
183 with st.sidebar:
184     st.header("⚙️ Content Settings")
185
186     brand_name = st.text_input("Brand Name", placeholder="e.g., TrendyTech")
187     purpose = st.text_input("Campaign Purpose / Topic", placeholder="e.g., Launching new product")
188
189     sample_text_input = st.text_area(
190         "Sample Brand Texts (one per line)",
191         placeholder="Example:\nSmart Living, Simplified\nInnovation at your fingertips\nUpgrade your home effortlessly",
192         height=175
193     )
194     sample_texts = [s.strip() for s in sample_text_input.split("\n") if s.strip()]
195
196     content_type = st.selectbox(
197         "Content Type",
198         ["Social Media Post", "Email", "Tagline", "Ad Headline", "Blog Intro"]
199     )
200
201     generate_btn = st.button("🚀 Generate Content")
202
```

Activity 3: Displaying and Copying Output

This activity presents the AI-generated content to the user and provides a quick way to copy it for reuse or sharing.

Explanation

- **Displays generated text:** Shows the output in a readable text area for easy review.
- **Enables instant copying:** Uses the “ **Copy to Clipboard**” button powered by *pyperclip* for one-click copying.
- **Enhances user experience:** Makes it simple and efficient to access, edit, or reuse the generated content

Code snippet:

```
203 # OUTPUT SECTION
204 st.subheader("Generated Content")
205
206 generated_text = st.session_state.get("generated_text", "")
207
208 generated_text = st.text_area(
209     "Generated Output",
210     value=generated_text,
211     height=350,
212     placeholder="Your generated content will appear here..."
213 )
214
215 if generate_btn:
216     if not brand_name or not purpose or not sample_texts:
217         st.warning("⚠️ Please fill in all the fields to generate content.")
218     else:
219         with st.spinner("Learning brand voice and generating content..."):
220             brand_prompt = learn_brand_voice(sample_texts)
221             generated_text = generate_brand_content(
222                 brand_prompt, brand_name, purpose, content_type
223             )
224             st.session_state.generated_text = generated_text
225
226         st.rerun()
227
228 if st.button("📋 Copy to clipboard") and generated_text:
229     try:
230         pyperclip.copy(generated_text)
231         st.success("✅ Text copied to clipboard!")
232     except Exception as e:
233         st.error(f"❌ Could not copy text: {e}")
234
```

Activity 4: Implementing a Feedback Button for Content Revision

Adds a feedback feature that lets users suggest edits, with the AI revising content in- This activity adds an **interactive feedback feature** that allows users to suggest edits to the generated content, enabling the AI to revise it while preserving the brand's voice and style.

Explanation

- **Incorporates user feedback:** Accepts user suggestions for improving or modifying the content.
- **Generates revised content:** Sends a combined prompt (original content + feedback) to Gemini to produce an updated version.
- **Updates in-place:** Replaces the old content seamlessly in the output area, supporting iterative refinement.
- **Maintains brand consistency:** Ensures that revisions still align with the established tone, style, and messaging.
- **Enhances usability:** Allows fine-tuning for clarity, emotional impact, and overall effectiveness without starting over.

Code Snippet:

```
236 # FEEDBACK / CHANGES
237 st.subheader("Feedback / Edit Section")
238
239 feedback_text = st.text_area(
240     "Provide your feedback or suggested changes for the generated content:",
241     height=100,
242     placeholder="E.g., make it more playful, add more emojis, shorten paragraphs..."
243 )
244
245 if st.button("🔄 Apply Changes") and generated_text and feedback_text:
246     with st.spinner("Applying feedback and updating content..."):
247 >         feedback_prompt = (
253             try:
254                 revised_response = model.generate_content(feedback_prompt)
255                 revised_text = revised_response.text.strip()
256                 st.session_state.generated_text = revised_text
257                 st.rerun()
258             except Exception as e:
259                 st.error(f"❌ Failed to apply feedback: {e}")
260
```

Milestone 5: Model Deployment

This milestone involves running and hosting the Streamlit app.

Activity 1: Running the Application Locally

To start the Streamlit app locally:

```
streamlit run app.py
```

This will launch the web interface in the default browser at:

```
http://localhost:8501
```

Code snippet:

```
You can now view your Streamlit app in your browser.
```

```
Local URL: http://localhost:8501
```

```
Network URL: http://192.168.0.218:8501
```

Activity 2: Online Access

You can optionally use **Localtunnel** or **Ngrok** on colab to expose your local Streamlit app to the internet for public access during demonstrations.

Example:

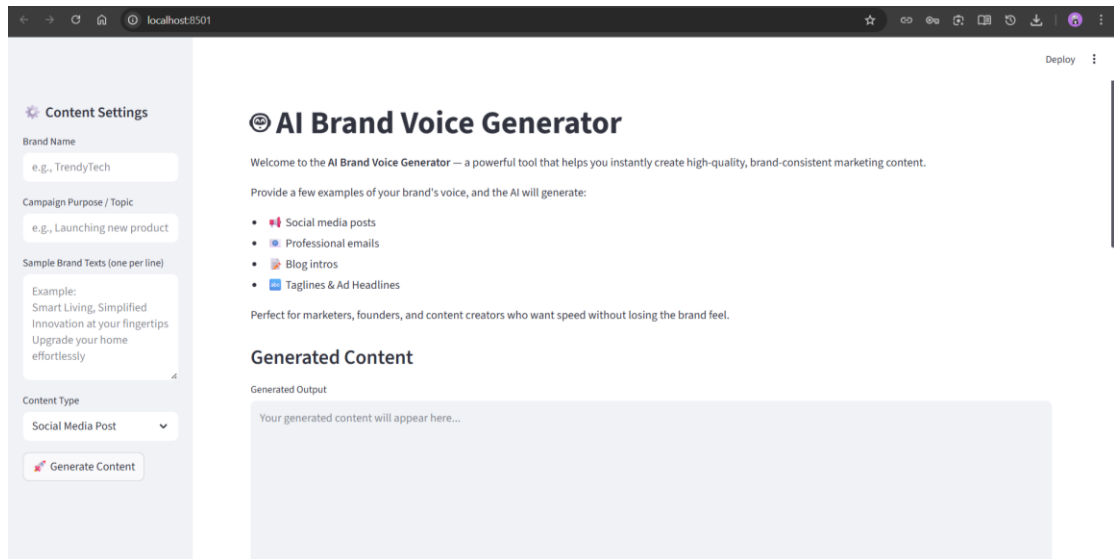
```
!npm install -g localtunnel
```

```
!streamlit run app.py & npx localtunnel --port 8501
```

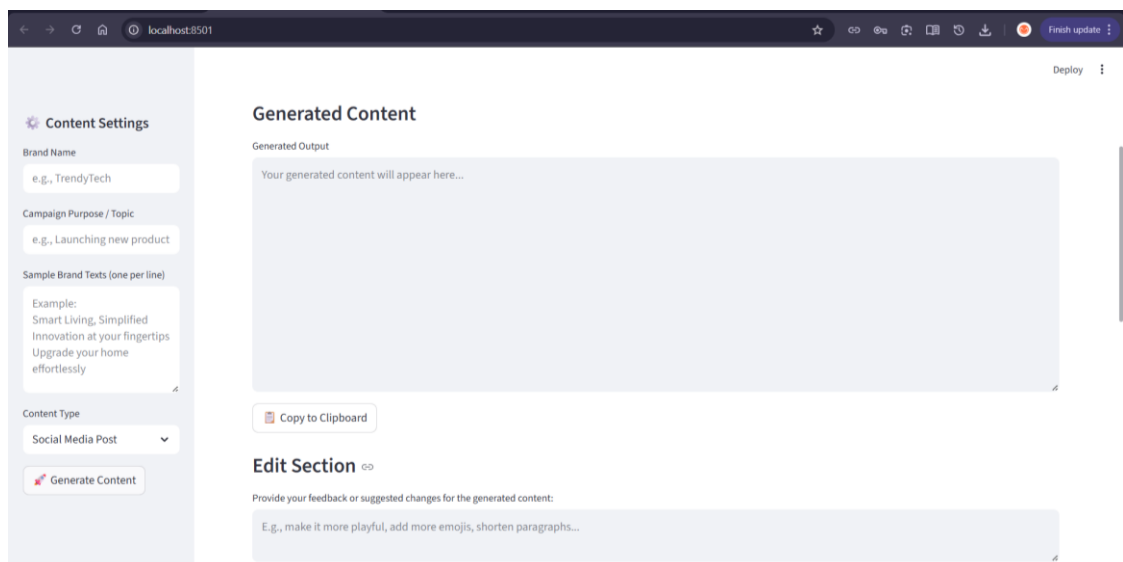
Output Preview

Output 1: Streamlit UI

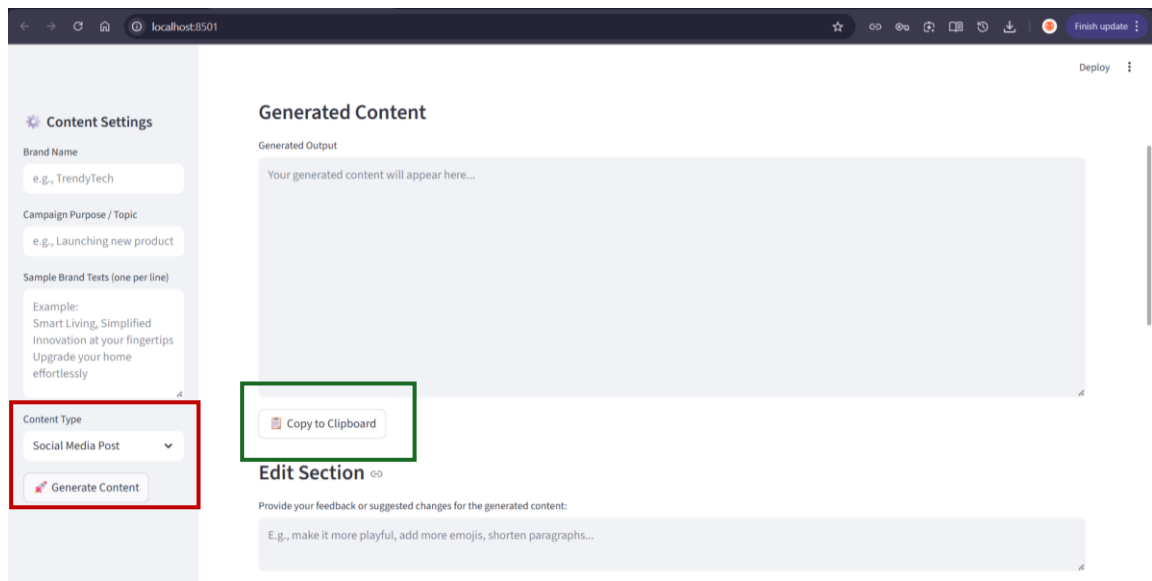
- Sidebar for brand input and configuration



- Central area displaying generated content



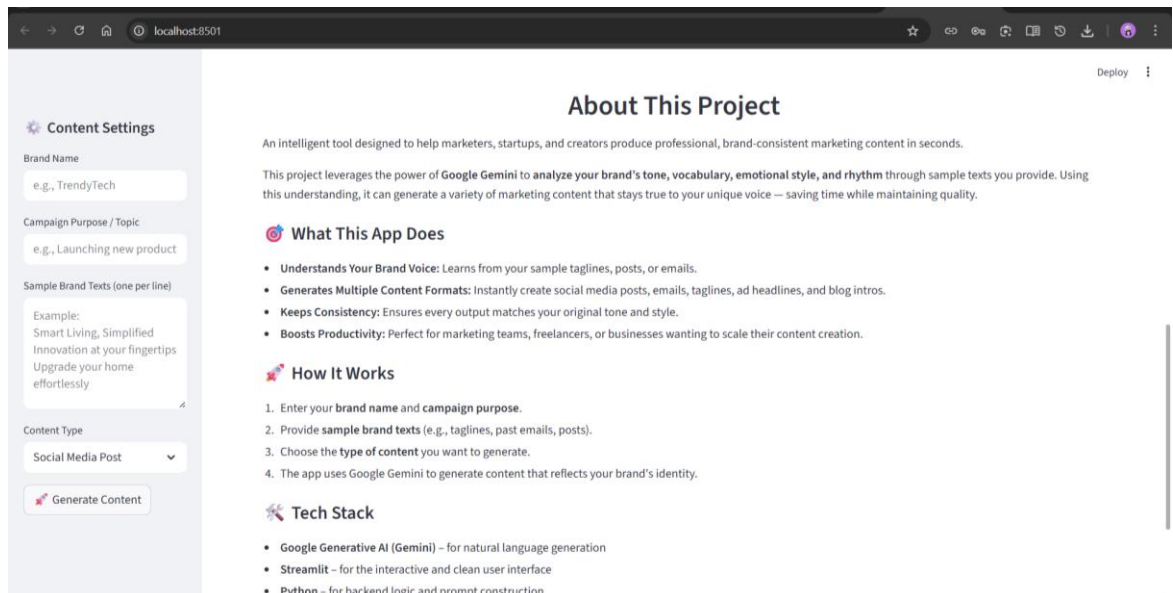
- Buttons for generating and copying text and feedback section.



Green colour highlights: Copy to Clipboard button

Red colour highlights: Generate Content button

- About Page



Output 2: Generated Content Examples

- Professionally written, brand-matched social media posts, emails, or taglines generated instantly by the Gemini model.

The screenshot shows a web application interface for generating content. On the left is a 'Content Settings' sidebar with the following fields: 'Brand Name' (Nike), 'Campaign Purpose / Topic' (Launching new shoes Nike Ai), and 'Sample Brand Texts (one per line)' (Every step tells a story of power, passion, and persistence.). Below these is a 'Content Type' dropdown set to 'Social Media Post' and a 'Generate Content' button. The main area is titled 'Generated Content' and displays three paragraphs of generated text for Nike Air Zoom. A 'Copy to Clipboard' button is located below the text. Below the generated content is an 'Edit Section' with a text input field for feedback, containing the text 'E.g., make it more playful, add more emojis, shorten paragraphs...'. The browser's address bar shows 'localhost:8501'.

Content Settings

Brand Name

Nike

Campaign Purpose / Topic

Launching new shoes Nike Ai

Sample Brand Texts (one per line)

Every step tells a story of power, passion, and persistence.

Content Type

Social Media Post

Generate Content

Generated Content

Generated Output

Feel the future with every stride! 🚀 Nike Air Zoom is here, engineered to redefine your run and elevate your game. Imagine floating on air, propelled forward by responsive cushioning that adapts to your every move. Whether you're conquering city streets or blazing trails, these shoes are designed to unleash your potential.

Experience the perfect fusion of innovation and style. Nike Air Zoom isn't just about performance; it's a statement. Feel the energy, embrace the comfort, and ignite your passion. Step into a world where limits are meant to be broken and goals are meant to be achieved. 🌟

Lace up and take flight! Explore the Nike Air Zoom collection now and discover the difference. Your journey to greatness starts here. 🏃‍♂️ #NikeAirZoom #RunFaster #JustDoIt #Innovation #NewRelease #Fitness #Performance #ShoeGame #NeverStop

Copy to Clipboard

Edit Section

Provide your feedback or suggested changes for the generated content:

E.g., make it more playful, add more emojis, shorten paragraphs...

- Providing feedback to generated output.

This screenshot shows the same web application interface as the previous one, but with feedback being provided. The 'Generated Content' section remains the same. In the 'Edit Section', the feedback text input field now contains 'remove emojis'. An 'Apply Changes' button is visible at the bottom of the feedback section. The browser's address bar still shows 'localhost:8501'.

Content Settings

Brand Name

Nike

Campaign Purpose / Topic

Launching new shoes Nike Ai

Sample Brand Texts (one per line)

Every step tells a story of power, passion, and persistence.

Content Type

Social Media Post

Generate Content

Generated Content

Generated Output

Feel the future with every stride! Nike Air Zoom is here, engineered to redefine your run and elevate your game. Imagine floating on air, propelled forward by responsive cushioning that adapts to your every move. Whether you're conquering city streets or blazing trails, these shoes are designed to unleash your potential.

Experience the perfect fusion of innovation and style. Nike Air Zoom isn't just about performance; it's a statement. Feel the energy, embrace the comfort, and ignite your passion. Step into a world where limits are meant to be broken and goals are meant to be achieved.

Lace up and take flight! Explore the Nike Air Zoom collection now and discover the difference. Your journey to greatness starts here. #NikeAirZoom #RunFaster #JustDoIt #Innovation #NewRelease #Fitness #Performance #ShoeGame #NeverStop

Copy to Clipboard

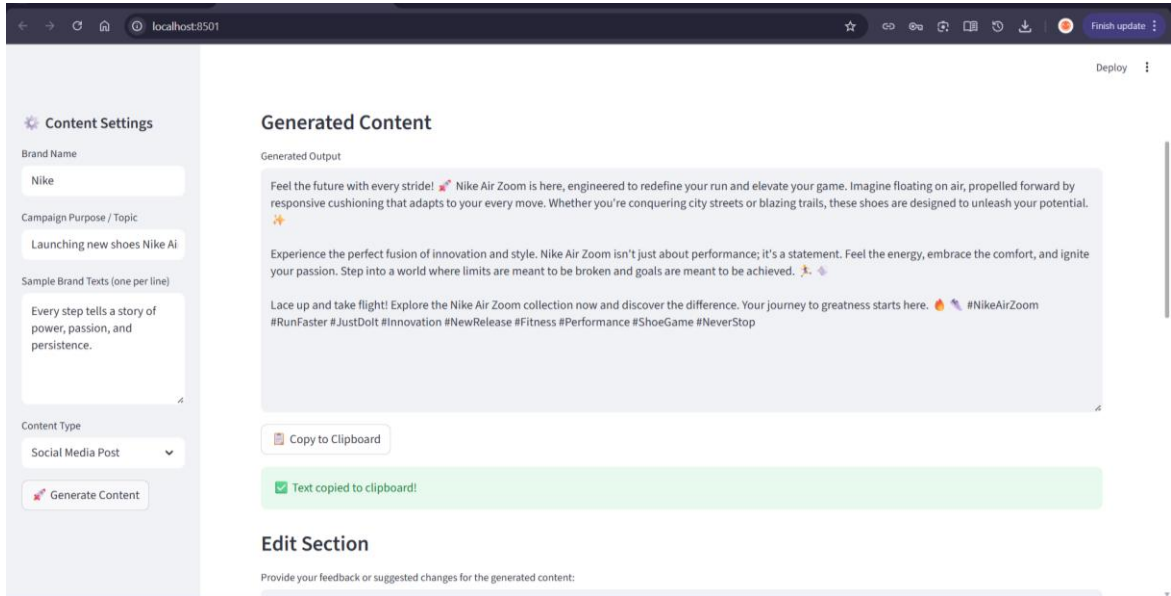
Edit Section

Provide your feedback or suggested changes for the generated content:

remove emojis

Apply Changes

- Copying the generated text with button.



Conclusion

The **AI Brand Voice Generator** revolutionizes the way brands create marketing content. By combining **AI understanding** of tone and style with **custom prompt templates**, it enables creators to scale their communication while maintaining brand consistency.

This project demonstrates the synergy between **Google Gemini's generative intelligence** and **Streamlit's interactive UI**, offering a practical, deployable solution for real-world content creation workflows.

With further enhancements such as multilingual support, campaign analytics, and fine-tuning this application can evolve into a complete AI marketing assistant for businesses of any size.